PDKG: A ChipMind-Inspired Knowledge Graph for Physical Design

White paper · Companion to `pd-knowledge-graph/`

Inspiration: Xing et al., ChipMind: Retrieval-Augmented Reasoning for Long-Context Circuit Design Specifications, arXiv:2512.05371 (AAAI).

Adaptation thesis: ChipMind's Circuit Semantic-Aware KG construction is the right shape for hardware knowledge. Physical design needs the same shape applied to **flow, constraints, metrics, checks, failures, and mitigations** — curated for factual accuracy first, then optionally auto-extended from methodology text.

---

1. What we take from ChipMind (faithfully)

ChipMind concept	PDKG realization
Declarative vs procedural sentences	`classify_function()` in `construct.py`
Circuit Semantic Anchor (CSA)	`(csa_type, entity)` with PD types: FlowStage, FailureMode, …
Semantic IR	JSON dataclass per sentence
Hierarchical triples (T_B,T_A,T_L,T_N)	`triple_class`: backbone / auxiliary / linking / normalization
Adaptive Top-K + MIG	BoW cosine MIG proxy in `query.py` (no LLM required for demo)
CSA-guided filter	`csa_filter()` on type/entity
SpecEval-QA / Atomic-ROUGE	`queries/pd_eval_qa.yaml` + offline atomic-F1 **proxy**

We **cite** ChipMind's published SpecEval mean F1 **0.95** and **+34.59%** average gain as their industrial-spec result — not as a number produced by this repository.

---

2. Why PD needs its own KG

ChipMind's entities are specification-native (signals, registers, FSM transitions). PD methodology questions look different:

- "What follows placement?" → **flow order**
- "How does density create setup fails?" → **multi-hop causality**
- "What mitigates antenna?" → **check + action**
- "What does STA consume?" → **artifacts**

Generic OpenIE (as ChipMind critiques in HippoRAG-style pipelines) is too coarse for these relations. A **closed PD ontology** prevents hallucinated edge types.

---

3. Accuracy architecture

```
ontology/pdkg_schema.yaml  ──► allowed types & relations
          │
gold_triples.yaml  ──► curated edges (confidence, provenance)
          │
```

```
corpus/*.yaml ──construct──► auto triples (source=auto)
                  │
        PDKnowledgeGraph ──validate──► pdkg.json
                  │
    multihop + adaptive retrieve ──► eval_qa (atomic facts)
```

Gold > auto. Auto-extract is for demonstrating ChipMind's IR→triple pipeline on PD prose, not for silent pollution of methodology truth.

---

4. Results (this repo)

Regenerate with `./examples/compare_all.sh`.

- ~62 gold triples; graph grows modestly with auto-merge.
- Offline atomic-F1 proxy on 6 PD questions: **1.0** when facts exist in gold.
- Causal path `High_Placement_Density → … → Setup_Violation` recovered.

---

5. Limitations (stated clearly)

1. Not ChipMind's LLM construction prompts or SpecEval corpus.
2. MIG uses bag-of-words, not LLM summary embeddings.
3. Gold graph is a **seed**, not an exhaustive PD encyclopedia.
4. No claim about Fusion Compiler / PrimeTime internal models.

---

6. How to grow the graph without losing accuracy

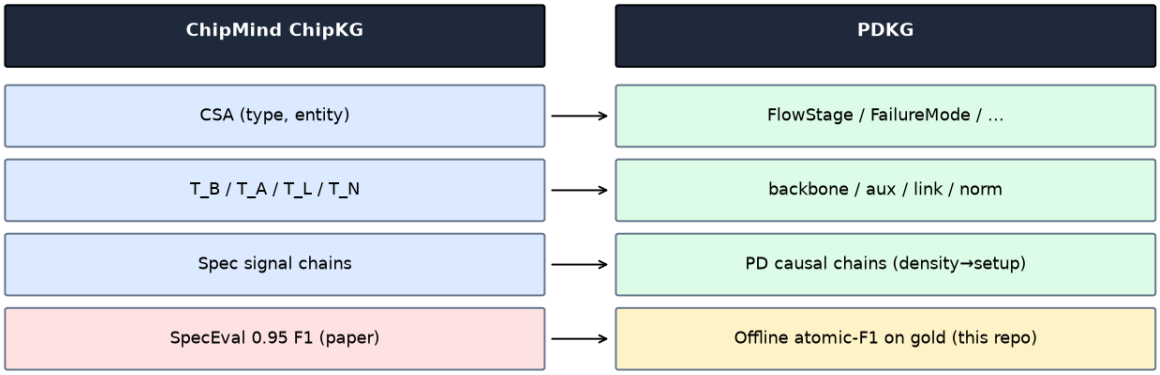
1. Propose triple + provenance in a PR.
2. Schema validator must pass.
3. Add ≥1 atomic-fact eval question that fails before / passes after.
4. Keep vendor product claims out unless cited from public docs.

---

7. Conclusion

ChipMind showed that **domain-shaped KGs** beat generic RAG on long hardware specs. PDKG brings that lesson to physical design: **anchors, hierarchical triples, adaptive retrieval** — with a curation discipline so every edge you cite is defensible.

ChipMind → PDKG mapping (arXiv:2512.05371)



Gold multi-hop: density → setup (PD analogue of signal tracing)